

PhD Training School in Poznan
Aleksandra Rutkowska

Foundations of Machine Learning and AI in Finance

From core ML concepts to the data realities and regulatory context of financial applications



Extended theory module — precedes Case Study 1: Financial Fraud Detection

Day 1 — Full Schedule

September 21, 2026 · 09:00–17:00 · Foundations of Machine Learning and AI in Finance

09:00–10:30	Part 0 — AI, ML & the paradigms of learning	<i>This deck</i>
10:45–12:15	Part A — AI, ML & the paradigms of learning	<i>This deck</i>
12:15–13:15	<i>Lunch break</i>	
13:15–14:45	Part B — Finance-specific challenges	<i>This deck</i>
15:00–16:15	Case Study 1 — Fraud: scale, taxonomy, sources	<i>Case Study 1 deck</i>
16:15–17:00	Exercises: dataset-landscape notebook + source-literacy briefing memo	<i>Notebook + handout</i>

Blocks 1–3 are this deck; Block 4 hands off to the Case Study 1: Financial Fraud Detection deck; Block 5 uses the companion notebook and handout.

Agenda for this deck

A

AI, ML & the paradigms of learning

What the terms actually mean, and the three ways a model can learn from data — each grounded in a finance example.

B

How a model actually learns

Classifiers, generalization, the bias–variance tradeoff, and the discriminative/generative distinction.

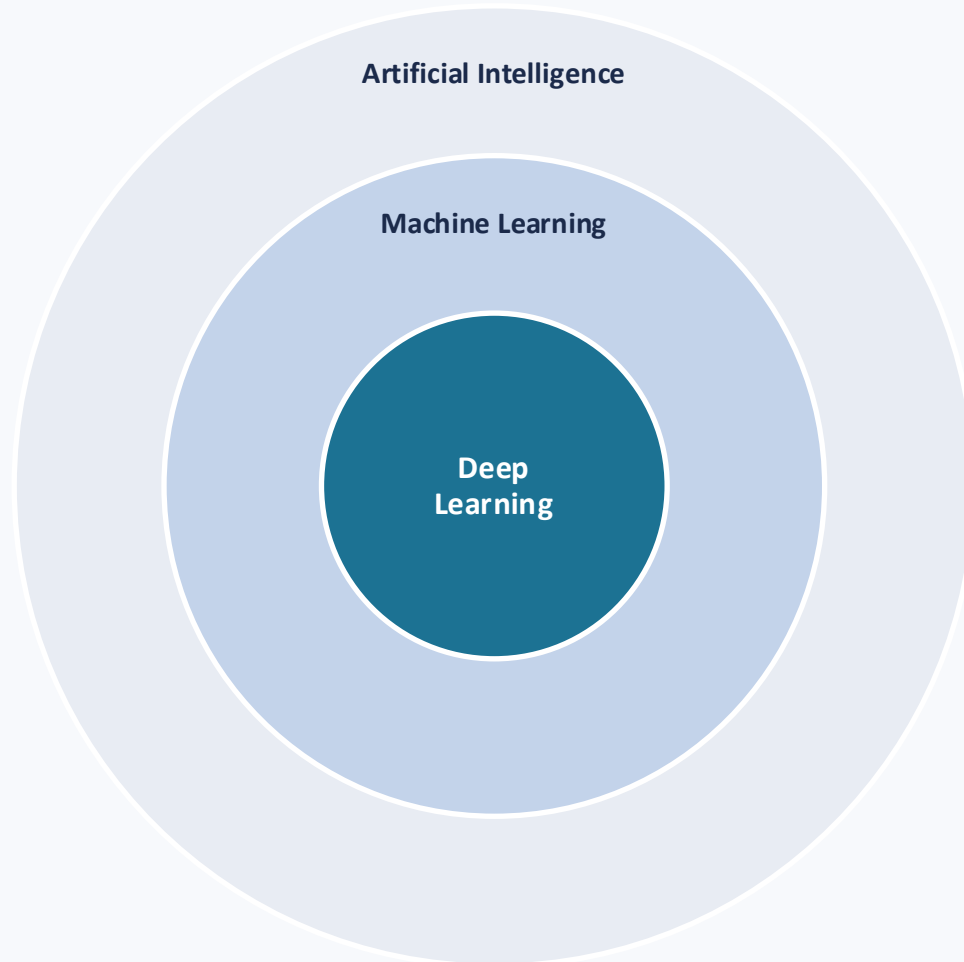
C

Finance-specific challenges

The data pitfalls, validation methods, and regulatory constraints that make financial ML different from a textbook problem.

AI, ML, and Deep Learning

Nested, not interchangeable — precision here avoids confusion later



Artificial Intelligence

Any system performing tasks that normally require human intelligence — includes rule-based expert systems, not just learning systems.

Machine Learning

Systems that improve at a task by learning patterns from data, rather than following hand-coded rules. This is our focus this week.

Deep Learning

ML using multi-layer neural networks — powerful with large data, but data-hungry and harder to interpret than the models we'll use for fraud.

Why finance is a natural fit for ML

Not hype — a specific set of properties financial problems tend to share

1

Abundant historical data

Transactions, prices, filings, and claims are recorded by default — a byproduct of how finance already operates.

2

Quantifiable outcomes

Default / no default, fraud / legitimate, profit / loss — labels exist naturally, unlike many other domains.

3

High cost of error

Even small accuracy gains translate directly into money, making the ROI case for ML unusually concrete.

4

Regulatory pressure

Compliance and reporting requirements push institutions toward systematic, auditable, model-driven decisions.

59%

of finance leaders report using AI in their finance function as of the 2025 Gartner AI in Finance Survey — up from 37% in 2023, though momentum has since slowed. Adoption is real, but far from universal or uniform.

Why finance is a natural fit for ML

Not hype —
a specific set of properties
financial problems tend to share

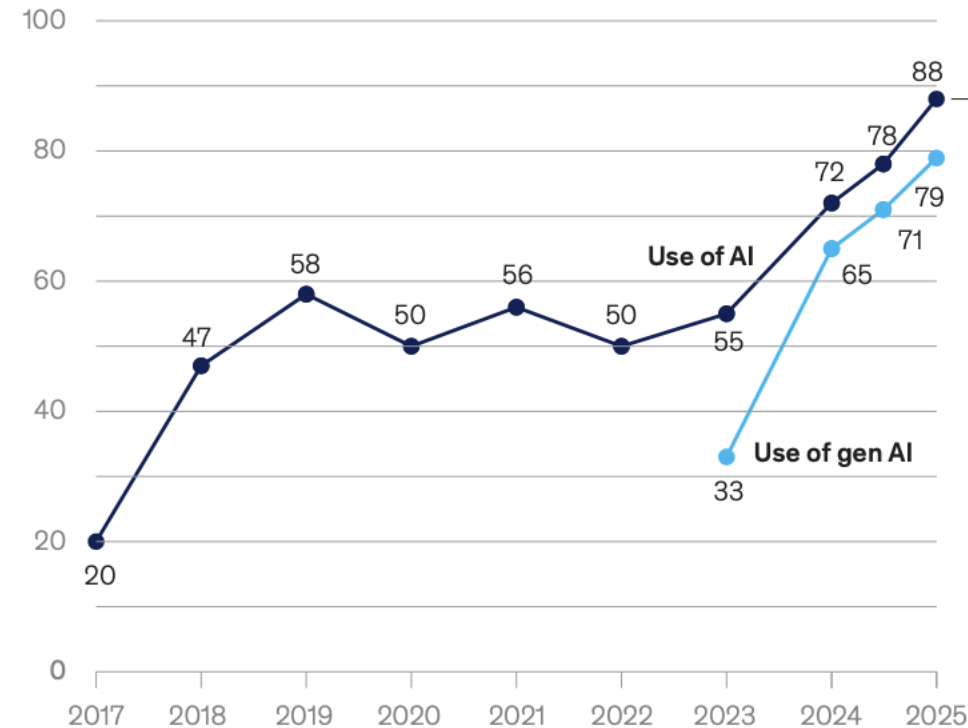
1

Qu

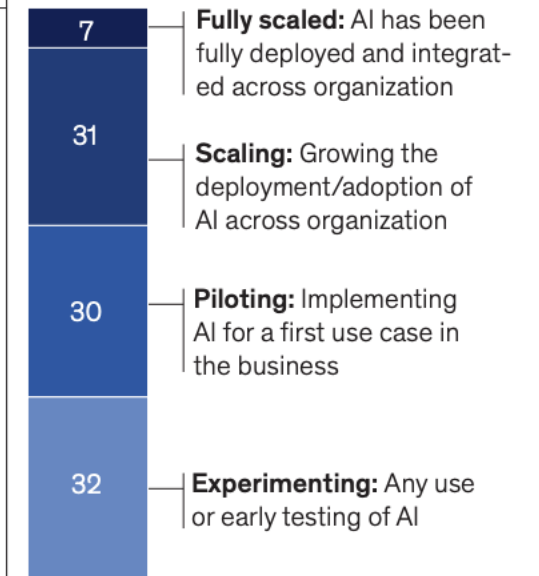
Reported use of AI in at least one business function continues to increase.

Use of AI by respondents' organizations, % of respondents

Organizations that use AI in at least 1 business function¹



Phase of AI use among organizations using AI in 2025



¹In 2017, the definition for AI use was using AI in a core part of the organization's business or at scale. In 2018–19, the definition was embedding at least 1 AI capability in business processes or products. From 2020, the definition was that the organization has adopted AI in at least 1 function, and in 2025, the definition was regular use of AI in at least 1 function.

Source: McKinsey Global Surveys on the state of AI, 2017–25

McKinsey & Company

Three ways a model can learn

Every technique in this course is an instance of one of these three paradigms



Supervised learning

Learn from labeled examples:
(input, correct answer) pairs.

"Is this transaction fraud?"



Unsupervised learning

Find structure in data with no
labels at all.

*"Which customers behave
similarly?"*



Reinforcement learning

Learn optimal decisions through
trial and error by receiving
rewards or penalties from its
environment.

*"How should I trade to
maximize return?"*

Supervised learning in finance

Learning a mapping from inputs to known, correct outputs

The setup

Training data: pairs of (features, label) — e.g. (transaction details, fraud/not-fraud).

Goal: learn a function that predicts the label for new, unseen inputs.

This is where classification and regression both live — it's the paradigm this whole course is built around, because fraud labels (eventually) exist.

Finance examples

- **Fraud detection:** transaction → fraudulent / legitimate (our case study).
- **Credit scoring:** applicant profile → will default / won't default.
- **Price prediction:** asset features → future return or price (regression, not classification).
- **Insurance underwriting:** policyholder profile → expected claim cost.

Unsupervised learning in finance

Finding structure when no one has labeled the answers

The setup

Training data: features only — no labels at all.

Goal: discover clusters, groupings, or a lower-dimensional structure hiding in the data.

Why it matters for fraud specifically: anomaly detection and One-Class SVM (covered in Case Study 1) borrow directly from this paradigm — modeling "normal," not "fraud."

Finance examples

- **Customer segmentation:** clustering account holders by behavior for marketing or risk tiers.
- **Anomaly detection:** flagging transactions or accounts that don't resemble the rest of the data — a fraud-detection strategy in its own right.
- **Dimensionality reduction (PCA):** compressing correlated risk factors into a handful of components for portfolio risk models.

Reinforcement learning in finance

Learning a policy by acting, observing outcomes, and adjusting

The setup

Training data: none, upfront — an agent takes actions in an environment and receives a reward signal.

Goal: learn a policy (a strategy) that maximizes cumulative reward over time.

Scope note: out of scope for this course's fraud case study, but worth knowing — it's the paradigm behind a real and growing slice of financial ML.

Finance examples

- **Algorithmic trading:** an agent learns when to buy/sell/hold to maximize returns net of transaction costs.
- **Portfolio optimization:** dynamically rebalancing allocations as market conditions change.
- **Optimal trade execution:** learning how to split a large order over time to minimize market impact.

Paradigms & Classifiers

Before we move on — can you answer these from what we just covered?

1

A bank wants to group customers into behavioral segments, with no predefined labels to train on. Which learning paradigm is this?

supervised · unsupervised · reinforcement

2

In the input $\rightarrow$ model $f(x) \rightarrow$ output picture, what exactly does "training" change? What stays fixed?

the function's parameters vs. its structure

3

True or false: a classifier can only output a hard class label (e.g. "fraud"), never a probability.

think about `predict()` vs. `predict_proba()`

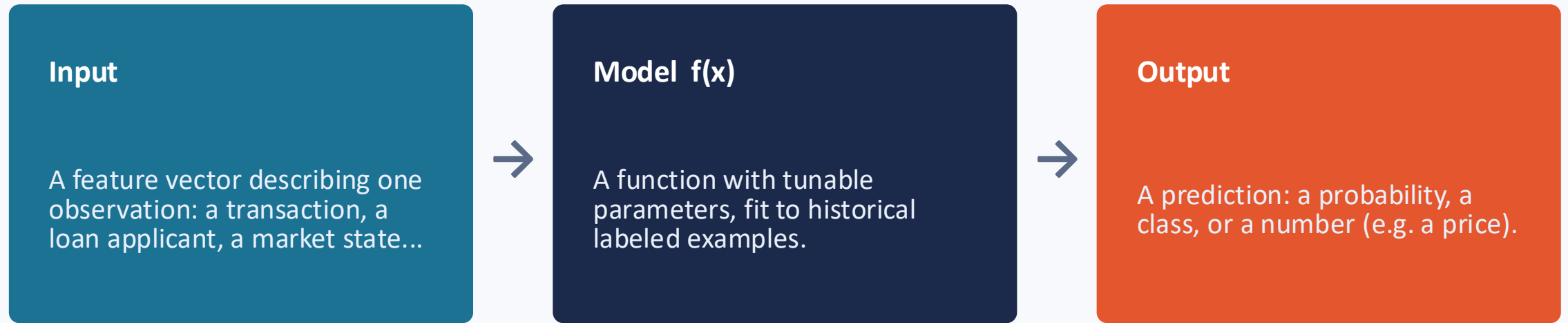
PART B

How a Model Actually Learns

Classifiers, generalization, and the tradeoffs every model makes

What is a model, really?

A function, fit to data, that maps inputs to a prediction



The learning problem: we don't hand-write the rule "if amount > \$500 and merchant is new → flag." We give the model thousands of labeled examples and let it find the pattern — that's the entire distinction between machine learning and a rule engine.

The goal is generalization, not memorization

A model that perfectly explains the past is not automatically useful for the future

Training set

The historical, labeled examples the model actually learns from. It can fit these as tightly as we let it.

Test set

Held-out data the model never saw during training — our best proxy for "tomorrow's data." This is the number that matters.

A model can score perfectly on the training set and fail completely on the test set. That gap is overfitting.

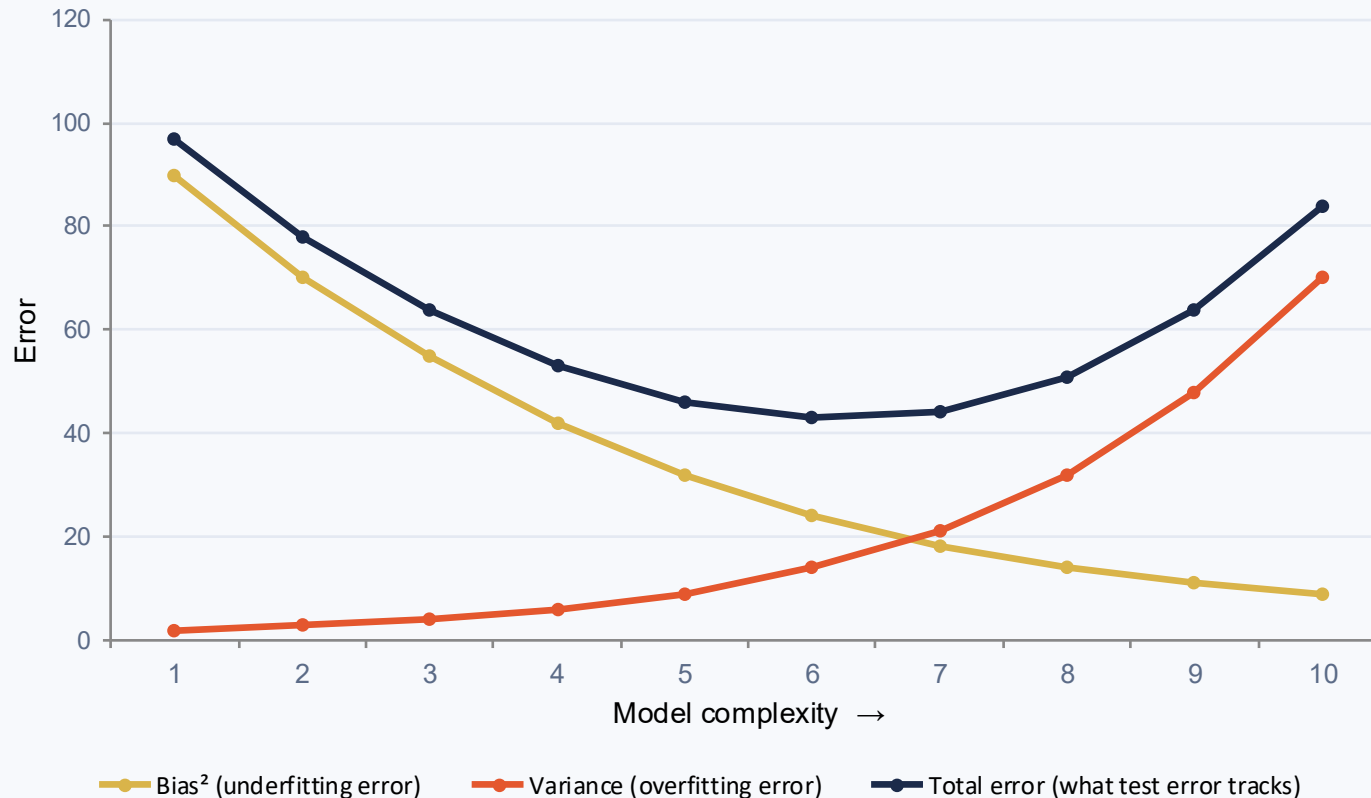
Why this matters for finance specifically

Markets and fraud patterns shift. A model that memorized last year's patterns instead of learning generalizable signal will fail on next year's data.

This is why, later this week, we'll be strict about train/test separation — especially once we start resampling data for imbalance, where it's easy to accidentally leak information across the split.

The bias–variance tradeoff

As a model gets more complex, two different sources of error move in opposite directions



Two kinds of error, defined

Bias — error from a model too simple to capture the real pattern, no matter how much data it sees. A straight line trying to fit a curve.

Variance — error from a model so flexible it fits the noise in this particular training sample. Retrain on a different sample and you'd get a very different model.

Total error (what the test set actually measures) is roughly bias + variance + irreducible noise. It's U-shaped because these two forces pull in opposite directions.

Sweet spot: total error is lowest around complexity 6 — where bias has fallen enough and variance hasn't yet taken over. That's the model we'd choose.

What this looks like in practice

The train/test error gap is your diagnostic

	Underfitting (high bias)	Good fit	Overfitting (high variance)
Training error	High	Low	Very low
Test error	High ($\approx$ training error)	Low ($\approx$ training error)	High ($\gg$ training error)
Train–test gap	Small	Small	Large
Typical cause	Model too simple; missing key features	Complexity matched to signal in the data	Model too complex for the amount of data
Fix	Add features, increase complexity, train longer	—	More data, regularization, simpler model, early stopping

What this looks like in practice

The train/test error gap is your diagnostic

	Underfitting (high bias)	Good fit	Overfitting (high variance)
Training error	High	Low	Very low
Test error	High ($\approx$ training error)	Low ($\approx$ training error)	High ($\gg$ training error)
Train–test gap	Small	Small	Large
Typical cause	Model too simple; missing key features	Complexity matched to signal in the data	Model too complex for the amount of data
Fix	Add features, increase complexity, train longer	—	More data, regularization, simpler model, early stopping

The diagnostic in one line: look at the gap between training and test error, not just the test error alone. A small gap with poor performance means underfitting — fix it by adding capacity. A large gap means overfitting — fix it by reducing capacity or adding data.

Two philosophies: discriminative vs. generative

Discriminative

"Learn the boundary between classes directly."

Examples: Logistic Regression, Decision Trees, Random Forest, XGBoost.

Strength: usually the strongest raw predictive performance — most of what we'll use in the case study.

Assumption: needs both classes well represented to learn a good boundary — which is exactly why imbalance hurts it.

Generative / one-class

"Learn what NORMAL looks like; flag whatever doesn't fit."

Examples: One-Class SVM, autoencoders, other anomaly-detection methods.

Strength: doesn't need many labeled positive examples — it only needs to model "normal" well.

Why it matters here: when the class of interest is 0.1% of the data, this reframing sidesteps the imbalance problem instead of fighting it.

How Models Learn

Same idea — check your understanding before we head into finance-specific challenges

1

A model scores 99% accuracy on training data but only 62% on test data. Is this underfitting or overfitting — and is bias or variance driving it?

recall the train/test gap diagnostic

2

Why might a bank choose a One-Class SVM over a Random Forest, even when the Random Forest scores higher accuracy on paper?

think about what each model needs to train, and what "accuracy" hides on imbalanced data

3

In one sentence: what's the difference between what a discriminative model learns and what a generative model learns?

"the boundary" vs. "what normal looks like"

PART C

Finance-Specific Challenges

The financial data pipeline

Financial ML draws on several structurally different data types



Transactional data

Payments, trades, claims — high volume, event-level, timestamped.



Market data

Prices, quotes, order books — continuous, high-frequency, noisy.



Structured company data

Financial statements, ratios, filings — low frequency, well-defined schema.



Text & alternative data

News, filings text, social sentiment, satellite imagery — unstructured, requires NLP/CV.

Data pitfalls unique to finance

Each of these has quietly ruined more than one published trading or credit model

1

Look-ahead bias

Accidentally using information that wouldn't have been available at prediction time (e.g. a restated financial figure).

2

Survivorship bias

Training only on companies/funds that still exist today silently excludes the failures — exactly the cases fraud/risk models most need.

3

Non-stationarity

Statistical properties of markets and fraud patterns drift over time; a model trained on 2020 data degrades by 2024.

4

Low signal-to-noise ratio

Especially in market data, most of the variation is noise — overfitting to noise is the default failure mode, not the exception.

...and four ways the problems themselves resist ML

Beyond data quality — the underlying prediction problems are structurally hard

1

Extreme class imbalance

Fraud, default, and failure events are all rare by definition — the focus of our case study.

2

Concept drift

Fraud schemes and market regimes both change; a static model decays without retraining.

3

Adversarial adaptation

Fraudsters — and sometimes competitors — actively probe and route around deployed models.

4

Weak / delayed labels

"Default" or "fraud" is often confirmed only after investigation, sometimes months later.

Validation: why plain k-fold isn't always enough

Financial data is often sequential — shuffling it before splitting can silently leak the future into the past

Standard k-fold CV

Randomly shuffles data into folds — implicitly assumes each row is independent of the others.

The problem for time-ordered data: a model can be validated on a fold that occurred BEFORE some of its training data — silently testing whether it can predict the past from the future.

Walk-forward (time-series) validation

Train on data up to time T, validate on data from T onward, then slide the window forward and repeat.



train (grows) validate (slides forward)

Interpretability isn't optional here

Financial ML operates under real regulatory constraints on explainability

Why regulators care

Model risk management (e.g. SR 11-7, US): banks must be able to explain, validate, and independently review models used for material decisions.

Credit decisions: adverse-action requirements in consumer lending mean a rejected applicant is often owed a reason, not just a score.

GDPR-adjacent expectations: automated decisions with legal effect carry expectations of meaningful information about the logic involved.

What this means practically

A slightly weaker but interpretable model (logistic regression, a shallow tree) is often preferred over a marginally stronger black box, precisely because of the accountability requirement.

Post-hoc explainability tools like SHAP and LIME let teams keep a complex model while still producing per-decision explanations — a compromise we'll revisit in the case study.

This is a genuine tradeoff, not a solved problem — keep it in mind every time we compare models purely on accuracy this week.

Accuracy vs. Explainability

This is where the theory gets personal — draw on your own experience, not just today's slides

- 1 Have you ever had a decision made about you — credit, insurance, hiring — that felt like a black box? What would have changed how it felt?
- 2 As a researcher, would you personally trade some model accuracy for interpretability? Under what conditions would you refuse to?
- 3 Who should carry the burden of explaining an AI-driven financial decision: the institution, the regulator, or the vendor who built the model?

The ML lifecycle in a financial institution

Model-building is one step of six — and rarely the one that determines success

↻ monitoring feeds back into data & retraining — this loop never truly closes



Where this course sits: steps 3–4 (model + validate) are where we'll spend most of our hands-on time this week, but every technique we cover — resampling, cost-sensitive learning, threshold tuning — is really a step-2/3 decision. Steps 5–6 (deployment, monitoring) matter just as much in practice, and we'll touch on them in the capstone.

Key takeaways



ML is a specific subset of AI: learning patterns from data, not hand-coding rules — and finance is unusually well-suited to it.



Supervised, unsupervised, and reinforcement learning each map to distinct financial problems; this course focuses on supervised classification.



Every model trades off bias and variance; the diagnostic is the gap between training and test error, not test error alone.



Financial data brings its own pitfalls (look-ahead bias, non-stationarity) and its own constraints (interpretability, regulation) beyond generic ML challenges.

Next: this afternoon's Case Study 1 — Financial Fraud Detection